

CE902 Machine Learning Code Review

Solved Questions and Additional Practice

This PDF contains complete solutions to the two given code-review questions and five new practice questions of the same type. Each answer identifies the main reliability issue, explains why it matters, and proposes a sound correction.

Section 1. Solutions to the two given questions

Question 1. Feature selection before cross-validation

Original code snippet:

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.feature_selection import SelectKBest
from sklearn.model_selection import cross_val_score

# Load dataset and create X and y
df = pd.read_csv('data.csv')
X = df.drop('target', axis=1) # all columns except target
y = df['target']

# Feature selection
selector = SelectKBest(k=10)
X_selected = selector.fit_transform(X, y)

# Fit in cross-validation using previously selected features
model = DecisionTreeClassifier(criterion='entropy', max_depth=9)
scores = cross_val_score(model, X_selected, y, cv=5, scoring='accuracy')

print('Cross-Validation_Accuracy:', scores.mean())
```

Main issue: The main issue is data leakage. SelectKBest is fitted on the full dataset before cross-validation, so each validation fold has already influenced which features were chosen.

Why this matters: This makes the reported cross-validation accuracy optimistically biased. The model is indirectly exposed to information from the validation folds during training, so performance on truly unseen data will usually be worse than the reported score. Feature selection may also become unstable, because a different subset of features may be optimal inside each fold.

Recommended fix: Wrap feature selection and the classifier inside a scikit-learn Pipeline and cross-validate the pipeline, not the already transformed data. That forces SelectKBest to be fitted only on the training portion of each fold. A good improvement is to use StratifiedKFold for classification and set random_state for reproducibility. If feature count or k must be tuned, tune it inside an inner CV loop rather than once on the full dataset.

Safer implementation:

```
from sklearn.pipeline import Pipeline
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.tree import DecisionTreeClassifier

pipe = Pipeline([
    ('select', SelectKBest(score_func=f_classif, k=10)),
    ('clf', DecisionTreeClassifier(criterion='entropy', max_depth=9, random_state=42))
])

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(pipe, X, y, cv=cv, scoring='accuracy')
```

Question 2. Hyperparameter tuning followed by re-evaluation on the same data

Original code snippet:

```
from sklearn.model_selection import cross_val_score, GridSearchCV
from sklearn.svm import SVC
import pandas as pd

# Load dataset
df = pd.read_csv('data.csv')

# Create X (all columns except target)
X = df.drop('target', axis=1)
# Create y (target column)
y = df['target']

# Perform grid search within cross-validation for parameter tuning
model = SVC()
param_grid = {
    'C': [0.1, 1, 10, 100],
    'gamma': [0.001, 0.1, 1],
}
grid_search = GridSearchCV(model, param_grid, cv=5, scoring='accuracy')
grid_search.fit(X, y)

# Select best model
best_model = grid_search.best_estimator_

# Fit classifier using the best parameters
# and assess its accuracy in cross-validation
scores = cross_val_score(best_model, X, y, cv=5, scoring='accuracy')

print('Cross-Validation_Accuracy:', scores.mean())
```

Main issue: The main issue is selection bias caused by reusing the same dataset both to choose the best hyperparameters and to estimate final performance. GridSearchCV tunes the model on all of X and y, then cross_val_score evaluates the chosen model again on the same data distribution.

Why this matters: The final accuracy is biased upward because hyperparameter tuning has already adapted the model to this dataset. In effect, the evaluation is no longer independent. A second practical issue is that SVMs are sensitive to feature scale, so omitting standardisation can further make results unreliable if features are on different ranges.

Recommended fix: Use nested cross-validation: an inner GridSearchCV tunes C and gamma inside each outer training fold, and the outer loop estimates generalisation performance. Also place StandardScaler and SVC in a Pipeline so scaling is learned only from the training split in each fold. If enough data exists, keep a final untouched test set for one last unbiased evaluation after model selection.

Safer implementation:

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import GridSearchCV, StratifiedKFold, cross_val_score
from sklearn.svm import SVC

pipe = Pipeline([
    ('scale', StandardScaler()),
    ('svc', SVC())
])

param_grid = {
    'svc__C': [0.1, 1, 10, 100],
    'svc__gamma': [0.001, 0.1, 1]
}

inner_cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
outer_cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

search = GridSearchCV(pipe, param_grid, cv=inner_cv, scoring='accuracy')
scores = cross_val_score(search, X, y, cv=outer_cv, scoring='accuracy')
```

Section 2. Five new similar practice questions and answers

Practice Question 1 - Scaling before cross-validation

Question snippet:

```
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

model = LogisticRegression(max_iter=1000)
scores = cross_val_score(model, X_scaled, y, cv=5, scoring='accuracy')
```

Answer - Main issue: Data leakage from preprocessing. StandardScaler is fitted on the full dataset before cross-validation.

Explanation: The means and standard deviations of the validation folds leak into the training folds. This makes the accuracy estimate slightly too optimistic, especially with small samples or strong distribution shifts.

How to fix it: Use Pipeline([('scale', StandardScaler()), ('clf', LogisticRegression(...))]) and cross-validate the pipeline. Then scaling is fitted separately inside each fold.

Practice Question 2 - Imputation and one-hot encoding outside CV

Question snippet:

```
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score

pre = ColumnTransformer([
    ('num', SimpleImputer(strategy='median'), num_cols),
    ('cat', OneHotEncoder(handle_unknown='ignore'), cat_cols)
])

X_pre = pre.fit_transform(X)
model = RandomForestClassifier(random_state=42)
scores = cross_val_score(model, X_pre, y, cv=5, scoring='accuracy')
```

Answer - Main issue: Preprocessing leakage. Imputation statistics and category mappings are learned from the entire dataset before validation.

Explanation: Validation-fold information influences medians and category encoding, so the score is no longer a clean estimate of unseen-data performance.

How to fix it: Put the ColumnTransformer and classifier into a Pipeline and cross-validate the pipeline. This fits imputers and encoders only on each training fold.

Practice Question 3 - SMOTE applied before cross-validation

Question snippet:

```

from imblearn.over_sampling import SMOTE
from sklearn.model_selection import cross_val_score
from sklearn.ensemble import GradientBoostingClassifier

smote = SMOTE(random_state=42)
X_bal, y_bal = smote.fit_resample(X, y)

model = GradientBoostingClassifier(random_state=42)
scores = cross_val_score(model, X_bal, y_bal, cv=5, scoring='accuracy')

```

Answer - Main issue: Severe leakage from oversampling before cross-validation.

Explanation: Synthetic minority examples are created using information from the whole dataset, so near-duplicates or related synthetic points can appear in both training and validation folds. This often inflates performance dramatically.

How to fix it: Use an imbalanced-learn Pipeline and place SMOTE inside the pipeline, so oversampling occurs only on the training split of each fold. Prefer stratified CV and metrics such as F1, recall, or PR-AUC for imbalanced data.

Practice Question 4 - PCA fitted before cross-validation

Question snippet:

```

from sklearn.decomposition import PCA
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import cross_val_score

pca = PCA(n_components=20)
X_pca = pca.fit_transform(X)

model = KNeighborsClassifier(n_neighbors=5)
scores = cross_val_score(model, X_pca, y, cv=5, scoring='accuracy')

```

Answer - Main issue: Data leakage from dimensionality reduction.

Explanation: PCA extracts components using variance structure from the full dataset, including validation folds. The transformed representation therefore contains information that should not be available during training.

How to fix it: Place PCA and KNN in a Pipeline and evaluate the pipeline with cross-validation. If the number of components should be tuned, do so in nested CV.

Practice Question 5 - Test-set reuse during model selection

Question snippet:

```

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

best_score = -1
for depth in [3, 5, 10, None]:
    model = RandomForestClassifier(max_depth=depth, random_state=42)
    model.fit(X_train, y_train)
    score = model.score(X_test, y_test)
    if score > best_score:
        best_score = score
        best_model = model

```

Answer - Main issue: The test set is being used for hyperparameter tuning.

Explanation: Because max_depth is chosen by looking at test-set accuracy, the test set is no longer independent. The final score will be over-optimistic and cannot be trusted as a final generalisation estimate.

How to fix it: Split the data into training and test only once, tune hyperparameters using CV on the training set, and reserve the test set for a single final evaluation. Alternatively, use nested cross-validation.